

FeelsLike

Buildings that listen — the complete explanation, written for someone who knows nothing about the project

The project in three sentences

People in an office type ordinary complaints like *"it's stuffy in Conference Room B"*. FeelsLike turns each complaint into a precise, safe instruction for the air-conditioning, checks the effect in a physics simulation of the building (a "digital twin"), and drives the real system. In a 7-day simulated comparison on identical weather it used **26.6% less energy** than a typical fixed schedule while producing **zero minutes of comfort violations**.

Document state: 15 September 2026 · Repository: github.com/rakshit-737/FeelsLike (public) · Every number in this document comes from a file or a run in that repository. Where something is simulated, estimated or not yet built, it says so.

Contents

- | | | | |
|----|---|----|---|
| 1 | The problem, in plain words | 13 | Reinforcement learning — and why it isn't shipped |
| 2 | The words you need first | 14 | The honesty machinery |
| 3 | The idea in one picture | 15 | The hardware: a real zone in the loop |
| 4 | Follow one complaint, start to finish | 16 | Calibration: making the twin match reality |
| 5 | The digital twin (the physics) | 17 | Is it feasible? Cost, deployment, payback |
| 6 | Understanding the complaint (NLP) | 18 | Limitations we state openly |
| 7 | Constraints: how a complaint becomes an instruction | 19 | The dashboard, tab by tab |
| 8 | The controller and its safety envelope | 20 | How the code is organised and how to run it |
| 9 | The experiment and the headline result | 21 | Where the project stands today |
| 10 | Comfort memory | 22 | Questions a judge will ask |
| 11 | Explain, What-if, Maintenance, Analytics | 23 | Glossary |
| 12 | The Monitor tab, real data and pricing | | |

How to read this

Sections 1–4 give the whole story at a high level. Sections 5–13 open each part of the machine. Section 14 explains why the numbers can be trusted. Sections 15–18 cover the physical hardware and the real-world case. Sections 19–23 are practical: using it, its current state, likely questions, and a glossary. Status tags used throughout: **VERIFIED** works and was checked by a run; **BUILT** code exists and is tested but not proven on hardware yet; **PAUSED** deliberately waiting; **NOT DONE** not started.

1 The problem, in plain words

Most office buildings cool every room on a fixed timetable, set to a cold number such as 22 °C, "just in case". Nobody asks the people inside whether they are comfortable. Two things go wrong at once:

- **Energy is wasted.** Air-conditioning (HVAC) is the largest single electricity use in an Indian office — roughly 40–60% of the building's electricity, per the LBNL/BEE study cited in our feasibility pack. Over-cooling a room by a degree or two, all day, every day, is expensive.
- **People are still uncomfortable.** The same room is too cold for someone by the window and too warm for someone near the projector. The only feedback channel is a facilities ticket that takes hours or days — or a sweater, or a space heater under the desk, which wastes even more energy.

The obvious fix — "just set the thermostat warmer" — saves energy but makes people too hot. The other obvious fix — "add occupancy sensors and a timer" — helps with empty rooms but still cannot hear that *this* meeting in *this* room is stuffy right now.

What FeelsLike does differently

It treats the occupants as sensors. Anyone can say, in their own words, what feels wrong. The system turns that into a small, time-limited, safety-bounded adjustment for exactly that room, and otherwise runs the building as lean as it safely can. The goal is **efficiency without sacrifice**: lower energy *and* fewer uncomfortable minutes.

Why a hackathon cares

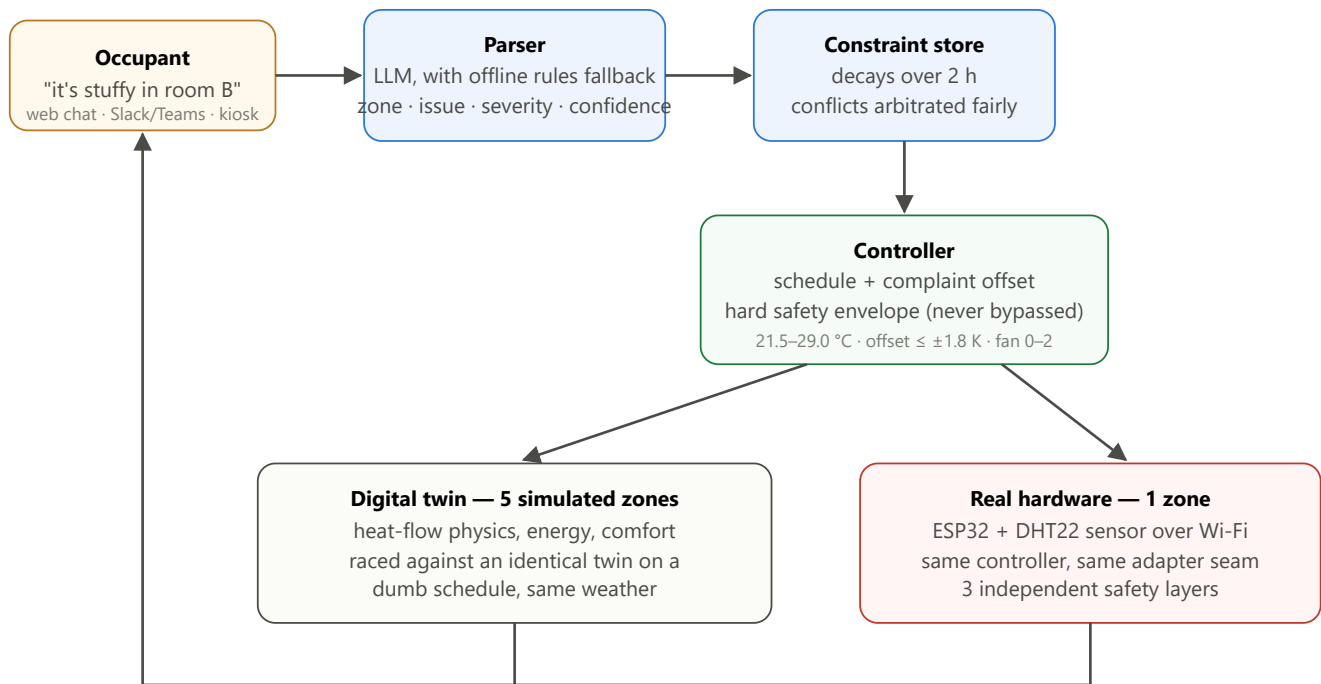
The problem statement asks for three things combined: a **digital twin** of a building, **optimisation** of how it runs, and **natural-language feedback** from people. FeelsLike is built around exactly that combination, and after round 1 the jury asked teams for more hardware, more original thinking, and a clearer view of real-world feasibility — which is why this project now includes a real physical sensor node, a cost and payback analysis, and a written list of its own limitations.

2 The words you need first

Only a handful of ideas are needed to follow everything else. A longer glossary is at the end.

Term	What it means here
HVAC	Heating, Ventilation and Air-Conditioning — the equipment that cools rooms and moves fresh air. In this project it mostly <i>cools</i> (it is modelled on a hot Indian climate).
Zone	One area the HVAC controls separately, such as a conference room. Our model building has five zones.
Setpoint	The target temperature you ask the HVAC to hold, e.g. 25.0 °C. A <i>higher</i> cooling setpoint means <i>less</i> cooling and less energy.
Vent / fan level	How hard the fan pushes air: 0 (off), 1, or 2. More fan brings more outdoor air (helps with stuffiness) but costs electricity.
kWh	Kilowatt-hour, the unit on an electricity bill. 1 kWh $\approx$ running a 1 kW appliance for one hour.
Comfort band	The temperature range counted as comfortable while people are present: 23.0 °C to 26.5 °C .
Violation-minute	One minute during which an <i>occupied</i> zone is outside the comfort band. This is how discomfort is measured. An empty room can never violate.
Digital twin	A computer model that behaves like the real building: given weather, people and HVAC commands, it computes how each room's temperature changes and how much electricity is used.
NLP	Natural Language Processing — getting a computer to understand free text. Here: turning "the cabin is freezing" into <i>zone = Cabin C, issue = too cold, severity = 2</i> .
LLM	Large Language Model — an AI text model (we use one hosted by Groq). It is one component, with an offline fallback so the system never depends on it.
Constraint	The structured, time-limited instruction created from one complaint: "make Conference Room B about 0.4 °C cooler with more airflow, fading over two hours".
Controller	The decision logic that, every simulated minute, chooses each zone's setpoint and fan level.

3 The idea in one picture



Dashboard: live race, explanations, alerts, every number labelled by source

Figure 1. The whole system. Text goes in at the top left; the building responds; the dashboard shows what happened and why.

Two design choices matter more than any other:

1. **The controller is the only thing that writes to the building, and everything it writes passes a hard safety check.** Complaints, the AI, humidity and crowd adjustments can only *request* small offsets; they can never command a dangerous temperature.
2. **Every result comes from a fair A/B race.** Two identical twins run side by side on bit-identical weather and identical occupants. One uses FeelsLike, the other uses the typical fixed schedule. The only difference is the controller, so the difference in energy and comfort is caused by the controller.

4 Follow one complaint, start to finish

At 14:05 on a Tuesday, someone types into the dashboard: "*it's really stuffy in conference room b*". Here is everything that happens, in order.

#	Step	What happens
1	Privacy scrub	Phone numbers and e-mail addresses are removed <i>before</i> the text leaves the server, while zone names and temperatures are protected so the meaning survives.
2	Parse	The LLM reads the text and returns strict JSON: zone <code>zone_b</code> , issue <code>stuffy</code> , severity 2, confidence ~0.9. If the AI is unreachable or returns anything invalid, the offline rules parser answers instead, in about a millisecond. Measured live: ~1.9 s via the LLM.
3	Guardrails	An unknown zone is erased (the system never acts on an invented room). No zone named → it asks "which zone?" instead of guessing. Not a comfort complaint ("the projector is broken") → ignored. "No longer stuffy" → the zone's complaints are cleared.
4	Constraint	A constraint is filed: stuffy at severity 2 means -0.4 °C and fan +1 for Conference Room B, starting at full weight and fading by half every 45 minutes, gone after 2 hours.
5	Arbitration	If someone else just said room B is freezing, the two are combined by a weighted average (severity × confidence × recency) and marked CONFLICT with a human-readable explanation.
6	Control	On the next simulated minute the controller takes the normal schedule (25.0 °C for an occupied room), adds the -0.4 °C offset (capped at ±1.8), clamps to 21.5–29.0 °C, and raises the fan.
7	Physics	The twin computes the room's new temperature and the extra electricity the fan and cooling use.
8	Hardware	Conference Room B is also the real zone. Its new fan level is sent to the ESP32 in the reply to its next reading, within ~2 seconds. VERIFIED on the bench: the fan channel lit up and the board echoed <code>fan=1</code> two seconds later.
9	Record	A decision record is logged: what changed, why (<code>reason_code</code>), estimated energy and comfort effect. The feed shows the parsed chip, latency, and whether AI was used.
10	Learn	If room B is reported stuffy around the same hour on two different days, comfort memory will start pre-applying a gentle fix 30 minutes early.

The point of this walkthrough

A casual sentence becomes a bounded, reversible, explainable action in about two seconds — and at no step can a misunderstanding produce an unsafe command. The worst case of a bad parse is a clarifying question or a small offset that fades on its own.

5 The digital twin (the physics)

The building

The twin models a 520 m² office floor with five zones. Each has its own size, orientation (which affects sunshine), heat-storage capacity, insulation, cooling capacity and occupancy pattern.

Zone	Name	Faces	Area m ²	Thermal mass C (MJ/K)	Envelope UA (W/K)	Cooling capacity (W)	People pattern
A	Open Office A	North	220	3.2	500	16,000	office (up to 24)
B	Conference Room B	South	60	0.9	160	7,000	meetings 10–11, 14–15:30, 16:30–17:30
C	Cabin C	East	40	0.6	110	4,000	3 people, 9–19
D	Lobby D	West	90	1.4	300	8,000	lobby traffic
E	Cafeteria E	South	110	1.6	340	10,000	lunch surge

Room B is highlighted because it is the zone mirrored by the real hardware (Section 15).

The model: a room as a leaky bucket of heat

Each zone is a **lumped RC thermal model** — the same idea as an electrical circuit with a resistor (R) and a capacitor (C). Think of the room's air and furniture as a bucket that holds heat (the capacitance C). Heat leaks in through walls and windows from the hot outdoors (the conductance $UA = 1/R$). Sun, people and neighbouring rooms add heat. The air-conditioner removes heat. The temperature is how full the bucket is.

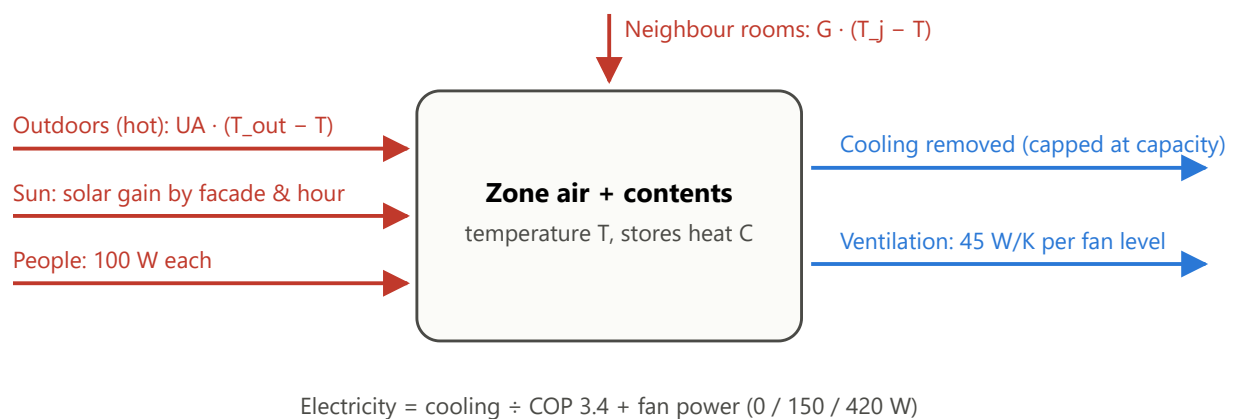


Figure 2. What flows into and out of one zone every simulated minute.

Every simulated minute (60 s), for each zone, the twin computes:

```

T_free = T + (60 s / C) × [ UA·(T_out - T)           heat leaking in through the envelope
                          + 45·fan·(T_out - T)        outdoor air brought in by ventilation
                          + solar_peak × solar_factor  sunshine (depends on facade and hour)
                          + 100 W × people            body heat
                          + Σ G_ij·(T_neighbour - T) ] heat through shared walls

q_cool = min( C·(T_free - setpoint) / 60 s , cooling capacity ) "perfect thermostat, finite
capacity"
T_new  = T_free - 60 s · q_cool / C
Power  = q_cool / 3.4 (COP) + fan power {0: 0 W, 1: 150 W, 2: 420 W}

```

- **Perfect thermostat, finite capacity:** the AC removes exactly the heat needed to hit the setpoint — unless that exceeds the unit's capacity, in which case the room drifts above setpoint. That is how real overload shows up.
- **COP 3.4** means one unit of electricity moves 3.4 units of heat out of the room — typical for an efficient split AC.
- **Neighbouring zones** share walls with conductances between 40 and 110 W/K, so a hot lobby warms the open office. All zones update simultaneously from the previous minute's values, so the order in which zones are computed never changes the result.
- **Weather** is synthetic, hot and deterministic: the same "seed" number always produces exactly the same weather. An optional live-weather fetch exists but never feeds the headline results.
- **Humidity** is tracked with a proper moisture balance but is *not* charged to energy — see Section 18.

Why this simple model and not a full CFD simulation?

A lumped RC model is the standard, literature-backed choice for control work: it runs thousands of times faster than real time on a laptop (the dashboard races a week in minutes), it has a handful of parameters that can be measured from a real room (Section 16), and it captures the effects that matter for a setpoint decision — thermal inertia, envelope loss, sun, people and capacity. It does not model air currents, radiant heat or stratification; those gaps are listed as limitations, not hidden.

6 Understanding the complaint (NLP)

What the parser must produce

Every message, whatever the language or wording, becomes the same small structure:

```
{ "is_comfort_complaint": true,
  "zone_id": "zone_b",           the single primary zone (or null)
  "zone_ids": ["zone_b"],       every zone named, for multi-zone messages
  "issue": "stuff",             too_hot | too_cold | stuffy | humid | drafty | other
  "severity": 2,                1 mild · 2 clear discomfort · 3 urgent
  "confidence": 0.9,           0 to 1
  "requires_clarification": false,
  "language": "en" }           en | hinglish | mixed
```

Two parsers, one contract

Primary: the LLM

A system prompt carries the exact schema, the list of real zones with their nicknames, anti-hallucination rules and worked examples (including Hinglish and a "projector is broken" negative). Any OpenAI-compatible provider works; the project currently uses **gpt-oss-120b on Groq's free tier**, measured at ~1.9 s per message. A 15-second timeout protects the live demo.

Fallback: the offline rules parser

A staged pipeline with no network and no AI:
normalise text → detect language → detect retraction
→ find zones (including typo tolerance) → detect intent → estimate severity → check context (appliances, outdoors) → compute confidence. It answers in about a millisecond, so the demo works on stage even with no internet.

Any failure of the LLM — no key, network error, malformed JSON, a schema violation — silently hands the message to the rules parser, and the feed shows which one answered.

What the parser handles

Kind of message	Example	Result
Plain complaint	"It's way too hot in Conference Room B"	too_hot, room B
Typos	"teh confrence rom is stuffy"	stuffy, room B
Hinglish / Tamil-English	"reception aur canteen dono me garmi hai"	too_hot, lobby + cafeteria
Multi-zone	"The lobby and cafeteria are too hot"	two constraints, one complaint id
Sarcasm / indirect	"love how I need gloves to type"	too_cold
Not about comfort	"the projector in room b is broken"	ignored
Appliance or outdoors	"the coffee machine is hot", "41 degrees outside"	ignored
No zone	"it's hot in narnia"	asks which zone — never guesses
Retraction	"no longer stuffy in reception"	clears that zone's complaints

How good is it — honestly

The benchmark has 70 labelled cases (30 development, 40 held-out). Because the team improved the parser by studying those cases, their scores read close to 100% and are **not** quoted as real-world accuracy. The number that is quoted comes from a separate **blind probe** of 20 sentences that were never used for development (Section 14 explains how it is kept honest):

Blind probe v2 (20 unseen cases, measured once)	Rules parser	LLM (auto mode)
Zone set exactly right	90%	90%
Complaint vs not-a-complaint	60%	70%
Issue right	65%	75%
Everything right at once ("exact triple")	45%	55%

Why 45–55% is still safe

The number that protects the building is zone extraction at 90% — and both zone misses added an extra real zone; neither invented a room. When the zone is unclear the system asks instead of acting. A wrong issue produces at most a ± 1.8 °C offset that fades within two hours. What still fails is predictable: fresh metaphors ("greenhouse vibes"), inverted sarcasm ("sponsored by the penguin exhibit") and implied heat in Hinglish. Those failures are committed in the repository, not deleted.

7 Constraints: how a complaint becomes an instruction

The effect table

Each issue and severity maps to a fixed setpoint offset and a fan change. Negative offset = cooler.

Issue	Severity 1	Severity 2	Severity 3	Fan
too_hot	−0.8 °C	−1.3 °C	−1.8 °C	0
too_cold	+0.8 °C	+1.3 °C	+1.8 °C	0
stuffy	−0.3 °C	−0.4 °C	−0.5 °C	+1
humid	−0.4 °C	−0.5 °C	−0.6 °C	+1
drafty	+0.3 °C	+0.4 °C	+0.5 °C	−1

Complaints fade: recency-weighted democracy, not a ticket queue

A complaint is strongest the moment it is made and loses influence over time:

$\text{weight} = \text{severity} \times \text{confidence} \times 0.5^{(\text{age} / 45 \text{ minutes})}$ and exactly 0 after 2 hours

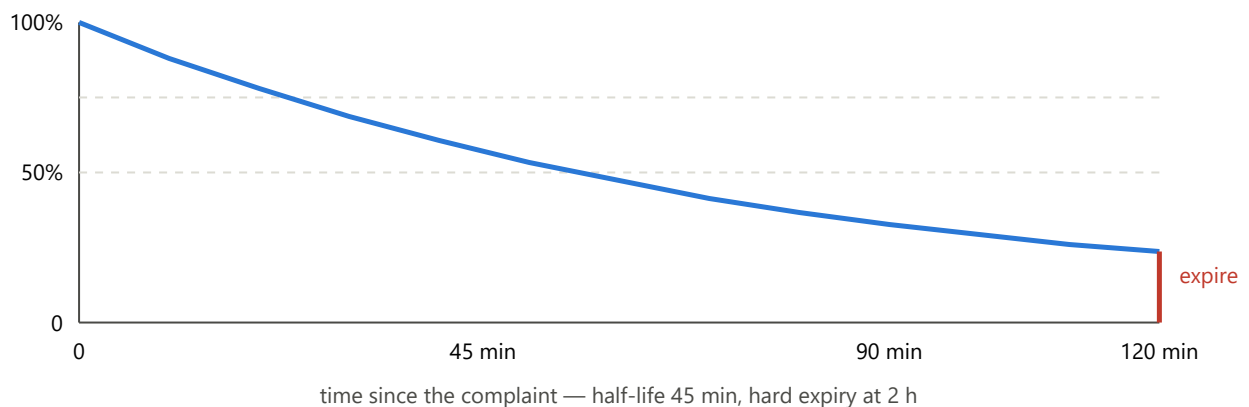


Figure 3. A complaint's influence over time. Re-reporting creates a fresh, full-strength constraint.

Why this design: if a room stays uncomfortable, people say so again and the new complaint counts in full. If the issue is fixed or the meeting ends, the adjustment quietly disappears and the building returns to its efficient schedule. Nobody's complaint wins forever, and nobody has to close a ticket.

When people disagree

If one person says room B is too hot and another says it's freezing, both constraints stay active and the zone's offset becomes their weighted average (by severity × confidence × recency). The dashboard marks the zone **CONFLICT** and shows a sentence such as "CONFLICT in zone: too hot (sev 2, 3m ago) vs too cold (sev 3, 1m ago) → severity-weighted compromise +0.4 °C". Because both decay, the compromise shifts naturally as the situation changes.

Human approval and all-clears

- In the **human_approval** safety mode a complaint is recorded and visible but moves nothing until an operator approves it. Rejected constraints stay in history but never influence control.
- An **all-clear** ("fine now in room B") expires that zone's active constraints immediately, using a separate `cleared` timestamp so the original complaint time is kept — comfort memory relies on it.

8 The controller and its safety envelope

The normal schedule

Without any complaints, the shipped **balanced** controller runs each zone on an occupancy-aware schedule:

Situation	Setpoint	Why
People present	25.0 °C	Inside the comfort band, warmer than the typical 22 °C, so far less energy
People expected within 30 minutes	26.0 °C	Pre-cool gently before arrival, with a ventilation flush
Room empty, nobody expected	28.5 °C	Setback — don't cool air nobody is breathing

The one formula that writes to the building

```
written setpoint = clamp( schedule(objective) + clamp(offset, -1.8 K ... +1.8 K) , 21.5 °C ... 29.0 °C )  
written fan      = clamp( schedule fan + complaint fan change + crowd boost , 0 ... 2 )
```

The offset is the *only* channel through which complaints, humidity or crowding can influence control, and it is clamped twice. Tests pile forty maximum-severity complaints onto one zone under every objective and a broken building, and the written command never leaves the envelope.

Extra adaptive terms

- **Humidity term** (max 0.6 K cooler): fires only when a zone has an active humid or stuffy complaint and high humidity — a DX coil only dehumidifies while it is cooling.
- **Crowd term** (max 1.0 K cooler): for zones above their design headcount, like the cafeteria at lunch.
- **Crowd fan boost**: needs both ≥ 20 people and $\geq 80\%$ of design density.

Objectives (operator's choice)

Objective	Occupied / pre-cool / empty
comfort	24.5 / 25.5 / 28.0
balanced (shipped)	25.0 / 26.0 / 28.5
cost, carbon	25.7 / 26.7 / 29.0
energy	25.9 / 26.9 / 29.0

Cost and carbon share one schedule on purpose: with a flat tariff and a flat grid-carbon factor, minimising rupees and minimising CO₂ are the same problem.

Five safety modes

Mode	What the controller does
automatic	Applies decisions immediately (default).
recommend_only	Computes and logs decisions but writes only the plain schedule.
human_approval	Queues complaint-driven changes until an operator approves them.
emergency_override	Ignores occupant constraints and drives every zone to the safe band.
maintenance_lockout	Freezes locked zones at their base schedule. A capacity alert locks its zone automatically, so the controller stops chasing a setpoint a saturated unit can't reach.

9 The experiment and the headline result

The setup

Seven simulated days (a Monday-to-Sunday week), weather seed 7, identical occupants, every controller started from the same state. Reproduce with `python -m scripts.demo_day`.

Energy over 7 simulated days (kWh) — lower is better



Comfort violation-minutes: static 16,328 · reactive 429 · **FeelsLike 0** · PPO 22

Figure 4. The headline comparison. Bar lengths are to scale.

Controller	kWh	₹ (flat ₹9/kWh)	kg CO ₂	Violation-min	Energy saved
Static 22 °C schedule — what buildings do today	722.4	6,502	512.9	16,328	—
Reactive thermostat at 24 °C	493.4	4,441	350.3	429	31.7%
FeelsLike (constraint-aware)	530.3	4,773	376.5	0	26.6%
FeelsLike with PPO agent	512.7	4,614	364.0	22	29.0%

How to read it

- **The static schedule wastes energy and still fails people:** 16,328 violation-minutes, almost all of them *over-cooling* — rooms colder than 23 °C while occupied.
- **A warmer dumb thermostat saves the most energy but breaks comfort:** 31.7% saved, with 429 minutes of people too hot. This is the tempting wrong answer.
- **FeelsLike saves 26.6% with zero violations.** It gives up about 5 points of savings versus the reactive thermostat to deliver comfort — that trade is the entire thesis: **efficiency without sacrifice**.

Say "in simulation"

Zero violations holds because the simulated cooling capacity is adequate and the thermostat model is idealised. Both sides of the race share that idealisation, so the *comparison* is fair — but a real building would not hit exactly zero. The feasibility maths deliberately assumes only 25% of these savings survive in reality (Section 17).

These four numbers are **frozen**: an automated check re-runs the week after every change, and the project refuses any change that moves them.

10 Comfort memory

Some discomfort is predictable. If Conference Room B gets stuffy every afternoon, why wait for someone to complain?

1. The memory scans real occupant complaints (never its own actions) for the same **zone + issue** reported on **at least 2 different days** at a similar time (within ± 1 hour).
2. Thirty minutes before that usual time, it pre-applies a **gentle** constraint — severity 1, confidence 0.5 — so the fix is small.
3. It announces the action in the feed: *"zone_b usually reports stuffy around 14:00 — pre-applying a gentle fix."*
4. If people already complained that day, it doesn't double up.

This turns a reactive system into a slightly anticipatory one, while staying transparent and bounded by the same envelope.

11 Explain, What-if, Maintenance, Analytics

Explainability: "Why did the system change this?"

Every material decision is logged as a structured record — never free-form AI text. A record holds the zone, the old and new setpoint and fan, what the plain schedule would have done, the occupancy, indoor and outdoor temperature, humidity, every constraint that contributed with its current weight, whether there was a conflict, the objective and safety mode in force, an estimated energy change and comfort change, a one-line human summary, and a `reason_code` from a fixed vocabulary (e.g. `precool`, `setback`, `constraint_applied`, `conflict_compromise`, `clamped`, `at_capacity`). The log keeps the last 500 and only records decisions that actually differ from the previous one for that zone.

What-if: counterfactuals that can't break the live building

An operator can ask questions like "what if occupancy rises 20%?". The engine takes a **clone** of the live twin and complaint store, applies the change, runs both the unchanged clone and the changed clone forward over the same horizon and weather seeds, and reports the difference with mean, spread and a 95% confidence interval across seeds. Results are labelled *measured* because the physics really ran. A fingerprint of the live state is checked before and after to prove nothing leaked. Eleven scenarios are built in:

Setpoint +1 °C	Setpoint -1 °C	Occupancy +20%	Occupancy -20%
Heat wave +3 °C	Monsoon +15 %RH	Cooling capacity 70%	Ignore all complaints
Complaints already expired	Objective: save energy	Objective: maximise comfort	

Maintenance: faults from evidence, not guesses

Four detectors watch the building: **capacity** (a unit pinned at 100% while its room stays hot), **sensor** (stuck, missing or impossible readings), **actuator** (a commanded change that produces no effect) and **recurring** (the same complaint again and again). None fires on a single bad reading — each needs sustained evidence (30–45 simulated minutes, with a short grace window) and quotes that evidence verbatim with numbers. Alerts are advisory: they never change control, except that a capacity alert locks its zone so the controller stops demanding cooling the unit cannot give.

Analytics

A bounded rolling sampler (every 15 simulated minutes) feeds a zone-by-hour comfort heatmap (occupied samples only, so an empty room at 3 a.m. is grey, not red), hourly and per-zone energy, complaint statistics (latency, confidence, clarification rate, by issue and zone) and controller activity (interventions, conflicts, clamps).

12 The Monitor tab, real data and pricing

The Monitor tab is the building-operator view: a zone selector, time windows from live to 7 days, 13 KPI cards, 9 charts with crosshair tooltips, click-to-drill-down, and an alerts table. Its defining rule is that **every value carries a badge saying where it came from**:

Badge	Meaning	Examples
SIM	The digital twin's own state	zone temperatures, occupancy, kWh
DERIVED	Computed from twin state by a documented formula	estimated CO ₂ , comfort score, power
HARDWARE	A real sensor on the bench	the ESP32 rig's temperature and humidity
REAL	A live external feed	Open-Meteo outdoor weather and air quality for VIT Chennai
HISTORICAL	A published dataset, not this building	UCI Occupancy Detection (office in Belgium, Feb 2015)
PREDICTED	The same controller run forward on clones	short-horizon forecast

- **CO₂ is estimated, and says so**: it uses the ASHRAE 62.1 single-zone mass balance with the twin's own ventilation and leakage rates. It is not a sensor reading.
- **Comfort score** (0–100) is a documented heuristic and is explicitly *not* the ASHRAE PMV comfort index.
- **Thresholds** for Normal / Warning / Critical come from one server-side table citing ASHRAE 55 and 62.1, WHO 2021 and EN 12464-1.
- **Noise** shows "not instrumented" because nothing in the system measures it — no number is invented.
- Simulated time and real wall-clock time are never drawn on the same axis.

Time-of-day electricity pricing (display only)

The verified Tamil Nadu commercial tariff (TANGEDCO LT-V, TNERC Order 6/2025) is ₹10.45/kWh at the margin, **+25%** in peak hours (06–10 and 18–22) and **–5%** at night (22–05). The dashboard re-prices the *same measured energy* at that tariff. The controller does **not** yet shift cooling to cheap hours — doing so would change energy use and invalidate the frozen results — so this view is labelled display-only.

13 Reinforcement learning — and why it isn't shipped

Reinforcement learning (RL) trains an agent by trial and error to maximise a reward. Here the reward penalises energy, discomfort and unmet complaints. The team trained a **PPO** agent (a standard RL algorithm) for **2 million steps**; its average reward improved from -91.8 to -86.8 and was still climbing when training stopped.

Comparison	PPO agent	Shipped rules controller
10 random single days — kWh per day	81.6	84.4
10 random single days — violation-min per day	2	0
7-day week — energy saved	29.0%	26.6%
7-day week — violation-minutes	22	0

The decision (milestone M4)

PPO saves slightly more energy but breaks the zero-violation result. Since "efficiency without sacrifice" is the whole point, the demo ships the explainable rules-based controller, and RL is shown honestly in the Experiments tab as a learning trajectory next to the measured comparison. A longer training run could change this decision; it would be re-measured, not assumed.

14 The honesty machinery

A claim is only as good as the process behind it. The project is built so that numbers cannot quietly drift or be inflated.

1 • The rotating blind probe

A test set stops being a fair test the moment you study its failures and fix them. The team caught this happening: after a parser rewrite the held-out split jumped from 55% to 100%, while a fresh set of 20 unseen sentences (probe v1) scored only 50%. Later, when probe v1's own failures were studied to guide fixes, **probe v1 was declared "burned"**: archived with the story written into it, and its later 80% re-score disowned. A new **probe v2** was written after all development, with the answers committed before measuring, and run exactly once per parser. The scoring script now **refuses** to run on a probe marked burned. That is why the quoted numbers are 45% / 55% — lower, but real.

2 • Frozen headline numbers

722.4 / 493.4 / 530.3 kWh and 16,328 / 429 / 0 violation-minutes are re-computed by tests and by a one-command pre-flight check. Any change that moves them is treated as a bug until proven otherwise.

3 • The capability gate

The presentation deck and reports are regenerated by scripts from the result files, so a number can't be typed into a slide and drift from the code. Claims about features (multi-zone, explainability, what-if, hardware, calibration, and others — 11 flags) are only written into the documents after the team has personally verified that feature. As of this document every flag is still off.

4 • Tests

534 automated tests pass. They cover data contracts between modules, the parser, constraint maths, thermal physics, controller safety under abuse, simulator determinism (same seed → bit-identical results), what-if isolation, the HTTP API including concurrency, the hardware bridge and its safety envelope, calibration, pricing, telemetry, and the ambient sensor node.

5 • Labels everywhere

Source badges on every dashboard value; *measured vs predicted* on every what-if result; a baked-in "SYNTHETIC SELF-TEST — NOT HARDWARE DATA" banner on calibration charts made from synthetic data; "uncalibrated" on sensor readings that haven't been cross-checked; and a written limitations register.

6 • AI used as an enabler, disclosed

The LLM is one measured component with an offline fallback. The feed marks every message that used external AI, and the privacy panel describes exactly what is sent. Physics, control, safety limits, the evaluation policy and the hardware design are engineering decisions the team owns and can defend.

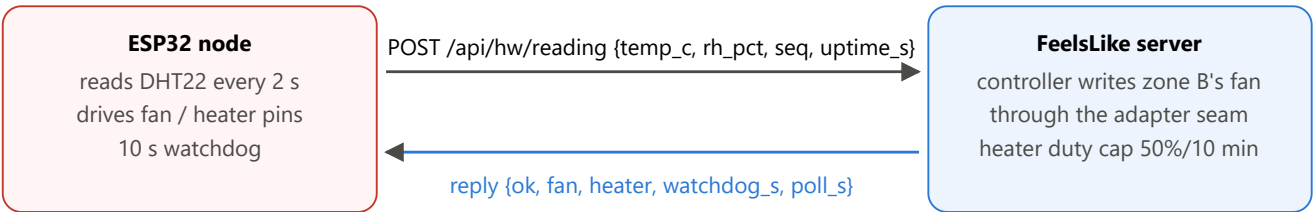
15 The hardware: a real zone in the loop

The round-1 jury asked for more hardware. FeelsLike now has a physical "shoebox zone" that mirrors **Conference Room B**.

What is on the bench

Part	Role	State
ESP32 dev board	Wi-Fi microcontroller; reads the sensor, drives the outputs, talks to the server. Classic ESP32 (ESP32-D0WD-V3) with a CP2102 USB chip.	VERIFIED
DHT22 sensor (GPIO 4)	Real temperature and humidity	VERIFIED
Blue LED (GPIO 16)	Stands in for the fan: off / dim / bright = fan 0 / 1 / 2	VERIFIED
Red LED (GPIO 17)	Stands in for the calibration heater	VERIFIED
L9110 driver, toggle switch, fuse	The power stage that would run a real fan and heater, with a physical kill switch	PAUSED — no fan yet; fuse rating unreadable
Arduino Uno + LM35	A second, wired sensor node measuring room air (Section 15.5)	BUILT — not yet run

How it talks to the server



One moving part: commands ride back on the reply to each reading. Plain HTTP over a phone hotspot — no broker to fail on stage.

Figure 5. The rig's wire protocol.

- **Same controller, same seam.** The rig is reached through the same adapter interfaces a real BACnet or Modbus building would use, and its adapters pass the exact conformance test the simulated ones pass. There is no special-case code path.
- **The twin is never overwritten by the real sensor.** Doing so would make the A/B race unfair and quietly change the headline numbers. Real readings are displayed, health-checked and logged for calibration instead.
- **Honest capabilities.** The rig has a fan but no cooling unit, so a setpoint command to it is *rejected and logged* ("rig is vent-only"), never faked.

Three independent safety layers

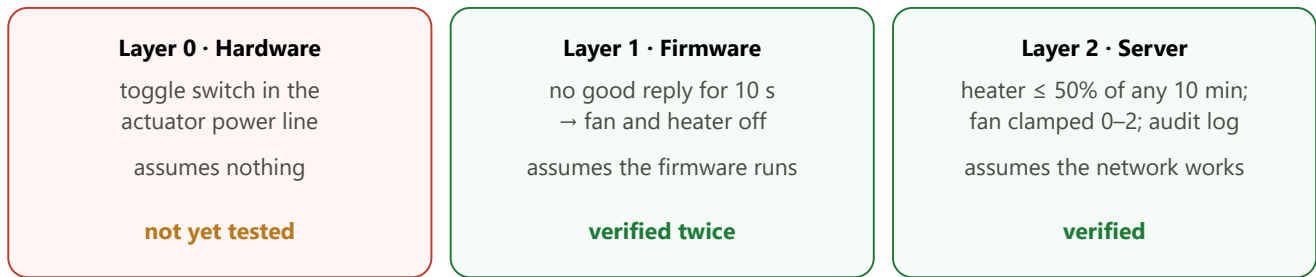


Figure 6. Each layer covers the failure the one above it cannot.

15.4 · What was verified on the bench (15 Sep 2026)

- **Sensing:** the node posted every ~2 s with every request accepted, reading about 30.5–31.2 °C and 77–80% humidity, sensor health clean.
- **Control loop:** "it is really stuffy in conference room b" → parsed by the LLM → the controller raised zone B's fan → the ESP32 echoed `fan=1` within 2 seconds → the blue LED lit.
- **Layer 2:** the heater was requested on; once it reached 50% duty the server forced it off (`duty_limited: true`) and the red LED went dark.
- **Layer 1:** the server was killed; the blue LED switched off on its own within the 10-second watchdog; when the server came back the node reconnected and the LED relit. Done twice and seen by eye.

15.5 · The wired ambient node (built, not yet run)

An Arduino Uno with an LM35 temperature sensor will measure the room air *around* the rig and send it over a USB cable, not Wi-Fi — the same shape as the wired RS-485/Modbus sensor buses in real buildings. A small bridge script forwards its readings. It posts to a separate, sensor-only endpoint whose reply can never contain a command and which can never overwrite the rig's reading (a test proves this). The Uno's internal voltage reference varies by up to ±10% between chips (about ±3 °C), so its readings are labelled **uncalibrated** until cross-checked against the DHT22 with a provided script. **PAUSED** until after the intermediate review.

Rehearsal without hardware

`python -m scripts.mock_node` runs a software ESP32 that behaves exactly like the firmware (including the watchdog) around a simulated box. The whole loop can be demonstrated even if the physical rig fails on stage — and it is presented as exactly that.

16 Calibration: making the twin match reality

A digital twin is only as good as its parameters. The strongest possible evidence is: *"we measured a real box and fitted our model to it, and here is the overlay of measured versus modelled temperature."*

How it works

1. A small heater of known power warms the closed box for a set time, then switches off, while the sensor logs temperature every 2 s.
2. The fitting script (`scripts/fit_rc.py`) searches for the insulation R and heat capacity C that make the RC model reproduce that temperature curve (least squares), using the heater's recorded on/off timeline.
3. It produces a measured-vs-model chart for the deck, plus the fit quality.

What is proven so far

- On synthetic data with known true values the fitter recovers R and C to within about 0.5%.
- An end-to-end test runs the whole hardware pipeline in software — including the server's 50% heater duty cap chopping the heating into on/off pulses — and still recovers R and C within 10%.
- `scripts/run_calibration.py` drives a real run with one command (about 65 minutes).

Status: PAUSED — needs a heater

The fit needs a heater of known wattage (for example a $10\ \Omega$ 10 W resistor on 5 V, or $47\ \Omega$ 10 W on 12 V). Until one is bought, no calibration result is claimed. Any chart made from synthetic data carries a permanent "NOT HARDWARE DATA" banner.

17 Is it feasible? Cost, deployment, payback

What one zone and one floor cost

Item	₹	Basis
One sensor node (ESP32, SHT31 sensor, PIR occupancy, IR blaster, power and enclosure)	≈ 1,050	retailer prices checked
20 nodes	21,000–22,000	
Gateway: Raspberry Pi 4 (or the building's existing PC at ₹0)	6,449	retailer price
Installation and commissioning (₹500/zone)	10,000	estimate
Spares (10%)	2,100	estimate
A 20-zone office floor (~2,000 m²)	≈ 40,000	

Two ways to install it

No building-management system (most Indian offices)

Sensor nodes plus **IR blasters** that send the same codes as the split-AC's remote (setpoint, mode, fan, on/off), and optional smart plugs. Honest limits: IR is one-way (the temperature trend acts as confirmation), split ACs change in 1 °C steps, and central plant such as air-handling dampers is out of reach.

Buildings with a BMS (BACnet or Modbus)

The controller's three calls — read state, write setpoint, write fan — map to standard points: per zone, five readings (temperature, humidity, CO₂, occupancy) and two writable points (cooling setpoint, fan speed). A systems integrator maps those points and runs the same conformance test the simulated and hardware adapters already pass.

Payback, with every assumption labelled

- **Measured:** 192.1 kWh saved over 7 simulated days on the 520 m² model.
- **Assumption:** every week is like that hot week → 10,016 kWh/yr on 520 m², scaled linearly to 40,065 kWh/yr for a 20-zone floor.
- **Verified tariff:** TANGEDCO LT-V commercial, ₹10.45/kWh at the margin.
- **Assumption, the big one:** how much of the simulated saving survives in a real building. We show 100%, 50% and 25% — and **defend only 25%**.

Savings realised ↓ · Tariff →	₹7.84 (–25%)	₹10.45	₹13.06 (+25%, peak)
100% of simulated	1.5 months	1.1	0.9
50%	3.0	2.3	1.8
25% — the defended case	6.0	4.5	3.6

Even at a quarter of the simulated savings and a tariff 25% lower, the ₹40k floor pays back in about six months. Carbon: 0.710 kg CO₂ per kWh (Central Electricity Authority database v21.0) → about 7.1 tonnes CO₂ a year per floor in the defended case.

18 Limitations we state openly

#	Limitation	What we do about it
1	The RC model ignores air currents between rooms, radiant heat and stratification; the thermostat is idealised	Only 25% of simulated savings are defended; hardware calibration exists to shrink that gap with evidence; "in simulation" is said aloud
2	Humidity is tracked but not charged to energy; simulated indoor humidity reads unrealistically high (~90%) because of a simplified coil approximation	A documented modelling choice that keeps the A/B comparison valid; the Monitor tab labels humidity a model limitation rather than hiding it; the fix is one constant and a team decision
3	Parser exact accuracy 45% (rules) / 55% (LLM) on unseen messages	Zone accuracy 90% with no invented rooms; ask-don't-guess; small, fading, clamped offsets; failures published
4	Sensors drift or die	Health is inferred (stale, stuck, out of range, impossible); a dead zone falls back to its schedule
5	IR control is one-way; relays wear	Solid-state switching for the fan; a command that changes nothing raises a maintenance alert
6	Network loss	Node watchdog turns actuators off within 10 s; the zone degrades to today's plain behaviour, never worse
7	One gateway is a single point of failure	Stateless nodes and a cheap cold spare; the building reverts to its existing schedule
8	Scale beyond one floor	Fine for a floor today; multi-floor needs a database-backed store and per-floor processes (designed, not built)
9	Time-of-day pricing not exploited	Shown as display-only; optimising it would need re-measuring the headline results
10	Zero violations is a simulation result	Both sides of the race share the idealisation, so the comparison is fair; not claimed for real buildings

19 The dashboard, tab by tab

Open `http://127.0.0.1:8000` after starting the server. A permanent badge states that the building is simulated. The top row holds the presenter's path; the second row holds depth for judges who want to dig in.

Tab	What it shows
Overview	The live race: energy bars and saved % for FeelsLike vs the dumb twin, comfort violations, money and CO ₂ tiles, floor plan with each zone's temperature, setpoint, fan and conflicts, and the complaint chat and feed.
Monitor	Operator view with source-badged KPI cards, charts, drill-down, alerts, and the physical-zone card showing the real sensor.
Live twin	Condition sliders (outdoor temperature, sun, humidity, occupancy, capacity), zone states, and the hardware rig card with measured-vs-twin chart and heater control.
Explain	The decision log — open any record to see exactly why a setpoint changed.
What-if	Run a counterfactual scenario on clones and compare baseline vs scenario with confidence intervals.
Demo	A guided script (Start / Next / Previous / Reset) that drives real state through the whole story in 2–4 minutes.
Complaints	Active and pending constraints, approve or reject.
Control	Objective, safety mode and zone locks.
Analytics	Comfort heatmap, energy series, complaint and controller statistics, time-of-day tariff view.
Maintenance	Active and resolved alerts with their numeric evidence.
Experiments	RL learning curve with the measured comparison, and the saved multi-seed what-if sweep.
Settings	AI disclosure, anonymity default, data retention, data export and redaction.

Complaints also arrive through **Slack or Microsoft Teams** (`POST /api/slack`) and a phone-sized occupant page.

20 How the code is organised and how to run it

The main parts

Path	Responsibility
sim/twin.py	The 5-zone RC digital twin: physics, energy, comfort, humidity
sim/controllers.py	Static, reactive, constraint-aware and RL controllers; the safety envelope
backend/parser.py	Complaint → structured JSON (LLM + rules)
backend/constraints.py	Decay, arbitration, explanations
backend/memory.py	Comfort memory
backend/decisions.py · whatif.py · maintenance.py · analytics.py	Explainability, counterfactuals, fault detection, analytics
backend/telemetry.py · external.py · tariff.py	Monitor data, real and historical feeds, time-of-day pricing
backend/hardware.py · adapters.py	The hardware bridge, sensor-only nodes and the adapter seam
backend/app.py	The FastAPI server: live simulation loop, all endpoints, dashboard
dashboard/	Single-page dashboard with no build step; works offline
hardware/	ESP32 and Uno firmware, wiring, safety model, bring-up runbook
evals/ · rl/ · scripts/ · tests/	Benchmarks and blind probe, RL training, tools, 534 tests
docs/	Deck, reports, feasibility pack, presentation script, this document

Run it

```
py -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

python -m scripts.demo_day	the 7-day evidence table
python -m evals.run_nlp_eval --rules	the NLP benchmark, offline
python -m evals.run_blind_probe --rules	the blind probe
python -m pytest tests -q	534 tests
python -m scripts.preflight	one-command pre-demo checklist
python -m uvicorn backend.app:app --host 0.0.0.0	the live dashboard on port 8000
python -m scripts.mock_node	a software ESP32, for rehearsal

No API key is required: without one, the offline rules parser keeps everything working. The `--host 0.0.0.0` flag is only needed when a real ESP32 must reach the laptop over Wi-Fi.

Licence

Code: Apache-2.0 or MIT, at the user's choice. Documentation: CC BY 4.0. Bundled third-party data keeps its own CC BY 4.0 licence and attribution (see [NOTICE](#)). If the hackathon's rules require different terms, those rules apply.

21 Where the project stands today

Item	State	Note
Digital twin, parser, constraints, controller, safety envelope	VERIFIED	tests + frozen numbers
7-day A/B result (26.6% saved, 0 violations)	VERIFIED	reproducible, in simulation
Comfort memory, multi-zone complaints, conflicts, retractions	VERIFIED	tests
Explain, What-if, Maintenance, Analytics, Monitor, pricing view	VERIFIED	API tests and live checks
RL agent trained and compared	VERIFIED	not shipped as controller, by decision
Blind probe v2 NLP scores	VERIFIED	45% / 55% exact, 90% zones
Real ESP32 + DHT22 sensing, control loop, watchdog, duty cap	VERIFIED	on the bench, LEDs as actuators
Feasibility pack: costs, deployment paths, payback, limitations	VERIFIED	assumptions labelled
Uno + LM35 wired ambient node	BUILT	code and tests done; paused until after intermediate review
Physical kill switch (safety layer 0) with L9110 power stage	PAUSED	no fan yet; fuse rating unreadable
Real fan and heater	NOT DONE	power supply and heater to buy
Hardware calibration overlay	NOT DONE	tools built and tested; needs the heater
Capability flags flipped, deck slides for hardware/feasibility	NOT DONE	only after team verification
Backup demo video, timed rehearsals	NOT DONE	

What to say about hardware today, accurately

"We have one real zone in the loop: a real sensor, and a real closed loop from a typed complaint to a physical output within two seconds, protected by a firmware watchdog and a server duty cap that we tested. The outputs are indicator LEDs today; the fan and heater stage is next."

22 Questions a judge will ask

How do you stop the AI from hallucinating a dangerous command?

The AI never writes a setpoint. It only produces a small structured label that is validated against a schema; unknown zones are erased; missing zones trigger a question; and every resulting offset is clamped to ± 1.8 K and the setpoint to 21.5–29.0 °C. If the AI fails, the offline parser answers.

Isn't 45–55% parse accuracy low?

It's the exact-everything score on unseen, deliberately hard sentences, measured once. Zone accuracy is 90% with no invented rooms, the system asks when unsure, and a wrong issue causes only a small offset that fades within two hours. We quote the hard number on purpose.

What happens when two people disagree?

Both complaints count, weighted by severity, confidence and recency; the zone is flagged CONFLICT with an explanation, and the compromise shifts as the complaints fade.

How real is the digital twin?

It's a standard lumped RC thermal model with envelope loss, sun by facade, people, ventilation, shared walls and finite capacity. Its parameters can be fitted to a real room, and we built and tested the tools to do that with our hardware.

Why not just use occupancy sensors and a timer?

That handles empty rooms, which our schedule also does. It can't hear that an occupied room is uncomfortable right now, and it can't settle a disagreement. FeelsLike does both, on top of occupancy-aware scheduling.

Why RC and not CFD?

Control needs a model that runs much faster than real time and can be calibrated from a few measurements. CFD is far too slow for that; its gaps are in our limitations list.

How does this attach to a real building?

Offices without a BMS: our nodes plus IR blasters that speak the AC remote's protocol. Offices with one: map five readings and two writes per zone to BACnet or Modbus points, and run our adapter conformance test.

What does it cost and when does it pay back?

About ₹1,050 per zone and about ₹40,000 for a 20-zone floor. Assuming only a quarter of our simulated savings survive, at the verified TANGEDCO tariff, payback is about 4.5 months.

What if a sensor drifts or dies, or the network drops?

Sensor health is inferred from its readings and a bad zone falls back to its schedule. If the network drops, the node's watchdog turns its outputs off within 10 seconds, so the room reverts to plain behaviour — never worse than today.

Would you trust it on a real building?

Three independent safety layers — a physical switch, a firmware watchdog and a server duty cap — plus a clamped control envelope and five safety modes including human approval. Two of the three layers are already tested on real hardware.

What did AI build, and what did you build?

AI assisted with code and powers the parser component, and its use is disclosed in the product. The physics model, the control law and its safety envelope, the arbitration rules, the evaluation policy (the blind probe and capability gate) and the hardware design are engineering decisions the team made and can defend.

Why is RL not your controller?

It saved slightly more energy but caused comfort violations. Zero violations is our thesis, so we ship the explainable controller and show RL as a measured trajectory.

23 Glossary

Term	Meaning
A/B race	Two identical simulated buildings on identical weather, differing only in the controller.
Adapter seam	A fixed set of interfaces (sensor, HVAC, occupancy, weather, notifications) through which the controller touches any building — simulated, hardware, or BMS.
Arbitration	Combining several active complaints for one zone into a single offset.
ASHRAE 55 / 62.1	Standards for thermal comfort and for ventilation and indoor air quality.
BACnet / Modbus	Common industrial protocols used by building-management systems.
BMS	Building-management system: the central computer that runs a large building's HVAC.
Blind probe	A set of test sentences never used during development, retired ("burned") once studied.
Calibration	Fitting model parameters (R, C) so the model reproduces measured data.
Capability gate	Flags that stop any feature from being claimed in the deck or report until verified.
Clamp	Force a value into a range: anything below the minimum becomes the minimum, above the maximum becomes the maximum.
COP	Coefficient of performance: heat removed per unit of electricity. Here 3.4.
DHT22 / LM35 / SHT31	Low-cost temperature (and, for DHT22 and SHT31, humidity) sensors.
Duty cycle	The fraction of time something is on. The heater is limited to 50% of any 10-minute window.
ESP32	A cheap microcontroller with built-in Wi-Fi.
Frozen numbers	The headline results, re-checked after every change and not allowed to move silently.
Half-life	Time for something to fall to half its value. A complaint's half-life is 45 minutes.
L9110	A small motor-driver chip that lets a microcontroller switch a fan or heater from a separate power supply.
Pre-cool / setback	Gently cooling before people arrive / letting an empty room warm up to save energy.
PPO	Proximal Policy Optimisation, a standard reinforcement-learning algorithm.
RC model	A room modelled like a resistor (insulation) and capacitor (heat storage).
Seed	A number that makes "random" weather reproducible: same seed, same weather.
TANGEDCO LT-V	The Tamil Nadu commercial electricity tariff used in the payback maths.
Violation-minute	A minute in which an occupied zone is outside 23.0–26.5 °C.
Watchdog	A timer that forces a safe state if expected messages stop arriving.

Sources for numbers in this document: `evals/results_energy.json`, `evals/results_blind.json`, `evals/results_blind_llm.json`, `rl/models/progress.csv`, `sim/twin.py`, `sim/controllers.py`, `backend/constraints.py`, `docs/FEASIBILITY.md`, `STATUS.md` and the verified bench session of 15 September 2026.